



Document Prepared By **Hamdan Ali**



NAME : HAMDAN ALI

WEEK 3 : Day 13

DAY 13

Create a function that takes a name as input and prints a greeting message on the webpage.

Task 13: Screenshoots Of Output

WEEK 3 - DAY 13 - FUNCTIONS, PARAMETERS & RETURN VALUES

Greeting Generator

Type a name, and a function builds the message

Try It

Enter a name and click "Greet Me" (or press Enter).

e.g. Hamdan

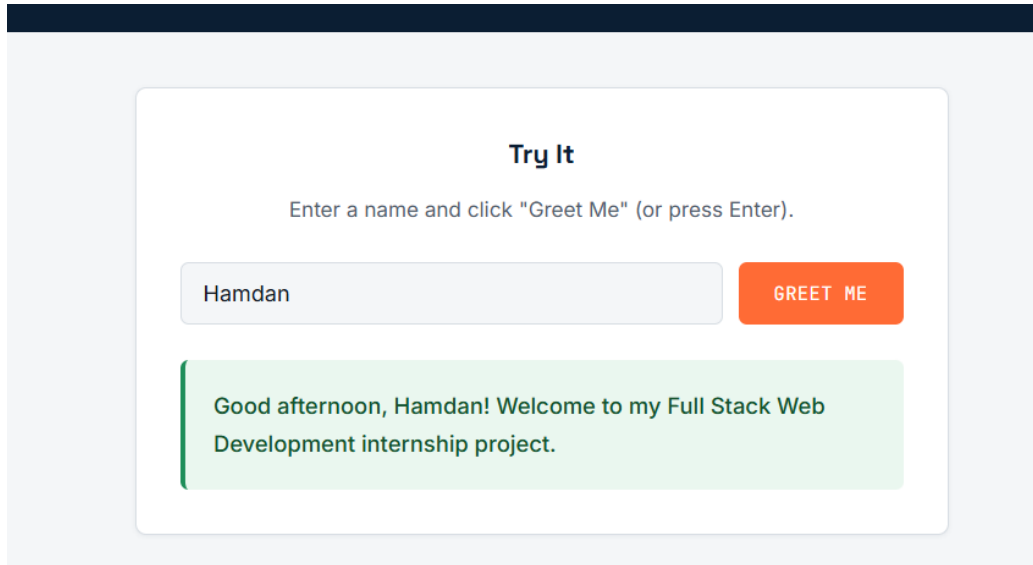
GREET ME

Your greeting will appear here.

What this does

Clicking the button calls `createGreeting(name)`, a function that takes the name you typed as a parameter and *returns* a complete greeting sentence — it doesn't touch the page directly. A separate function, `displayGreeting()`, takes that returned string and writes it into the page.

After Enetering Name :



EXPLANATION

DAY 13 — FUNCTIONS, PARAMETERS & RETURN VALUES

Building a Greeting Generator

Hamdan Ali · Week 3 · Full Stack Web Development Internship

What This Task Was About

Day 13 asked for a function that takes a name and prints a greeting on the webpage. That's a one-line description, but it's really testing three separate ideas at once — writing a function, passing it a parameter, and having it return something useful rather than just doing one fixed thing every time. So instead of writing one function that both builds the message and shoves it onto the page in the same breath, I split those two jobs apart, because that's closer to how a function is actually supposed to behave.

The page itself is simple: a text box, a "Greet Me" button, and a spot where the message appears. Nothing fancy on the surface — the actual work is in how the functions are organized underneath.

Separating "Build the Message" From "Show the Message"

The function the task is really asking for is `createGreeting(name)`. It takes a name as a parameter and returns a full sentence — it does not touch the webpage at all:

```
function createGreeting(name) {  
  if (!name || name.trim() === "") {  
    return "Please enter a name so I know who I'm greeting.";  
  }  
  
  const cleanName = formatName(name);  
  const timeGreeting = getTimeOfDayGreeting();  
  
  return `${timeGreeting}, ${cleanName}! Welcome to my Full Stack Web Development  
internship project.`;  
}
```

Keeping this function "pure" — meaning it only takes input and returns output, without reaching out and changing anything on the page — means it could be reused anywhere. It could feed a `console.log`, an alert, a different page, even a test file, without ever needing to know that a button exists. That separation is really what "return values" are for: handing a result back so something else decides what to do with it.

The part that actually writes to the page is a different function entirely:

```
function displayGreeting() {  
  const message = createGreeting(nameInput.value);  
  output.textContent = message;  
  output.classList.add("greeting-output--visible");  
}
```

This one reads the input field, calls `createGreeting()` to get the sentence, and only then updates the DOM. It's a small distinction, but it's the difference between a function that does one thing and a function that quietly does three things at once.

Two Small Helper Functions

`createGreeting()` leans on two smaller functions rather than doing everything itself. The first cleans up whatever the user typed:

```
function formatName(name) {  
  const trimmed = name.trim();  
  return trimmed.charAt(0).toUpperCase() + trimmed.slice(1).toLowerCase();  
}
```

This turns "hAMDAN" or " hamdan " into a properly capitalized "Hamdan" — trimming stray spaces and fixing whatever casing the user happened to type in. The second helper picks a time-appropriate greeting word:

```
function getTimeOfDayGreeting() {  
  const hour = new Date().getHours();  
  
  if (hour < 12) {  
    return "Good morning";  
  } else if (hour < 18) {  
    return "Good afternoon";  
  } else {  
    return "Good evening";  
  }  
}
```

This one is worth pointing out for a different reason: it takes zero parameters, but it still returns a value. That matters because it's easy to assume a function needs input to be useful — this one proves otherwise. It just checks the current time and hands back one of three strings.

Why Split It Into Four Functions Instead of One

The task could technically be done with a single function that grabs the input, checks it, and writes straight to the page. I split it into `createGreeting()`, `formatName()`, `getTimeOfDayGreeting()`, and `displayGreeting()` on purpose. Each one has exactly one job, each one is easy to test on its own by just calling it with different values, and if I ever needed to change how names are formatted, I'd only touch `formatName()` instead of digging through a much longer function to find the right lines.



It also made the parameters-and-return-values part of the task more obvious rather than less — every function here either takes something in, gives something back, or both, and none of them are doing invisible side jobs that aren't reflected in their name.

A Few Notes on What I Took From This

- A function that returns a value is more reusable than one that directly changes the page — it can be tested, reused, or displayed differently without being rewritten.
- Parameters don't have to be required to be useful — `getTimeOfDayGreeting()` takes none and still returns something meaningful.
- Splitting work into small, single-purpose functions made the code easier to read back later, not just easier to write the first time.
- Validating the parameter inside `createGreeting()` (checking for an empty name) meant the rest of the app never had to worry about that case again.

References Used

I used the W3Schools functions page for the exact syntax around parameters and return statements, and watched the linked video to see functions written and called in a live example.

Article: [w3schools.com/js/js_functions.asp](https://www.w3schools.com/js/js_functions.asp)

Video: youtube.com/watch?v=Jtc3j4ZNZEQ

Files Submitted

- `script.js` — the four functions described above (`createGreeting`, `formatName`, `getTimeOfDayGreeting`, `displayGreeting`).
- `index.html` — a page with a name field, a button, and a live output area where the greeting is displayed.

THANK YOU!